

NETWORKING IN KUBERNETES

Index Table

1	Why Kubernetes Networking Matters
2	Networking Basics in Kubernetes
3	Pod-to-Pod Communication
4	Services Explained & Ingress
5	Traffic Flow: End-to-End
6	Common Networking Pitfalls
7	Practical Demo Overview
8	Key Takeaways & What's Next

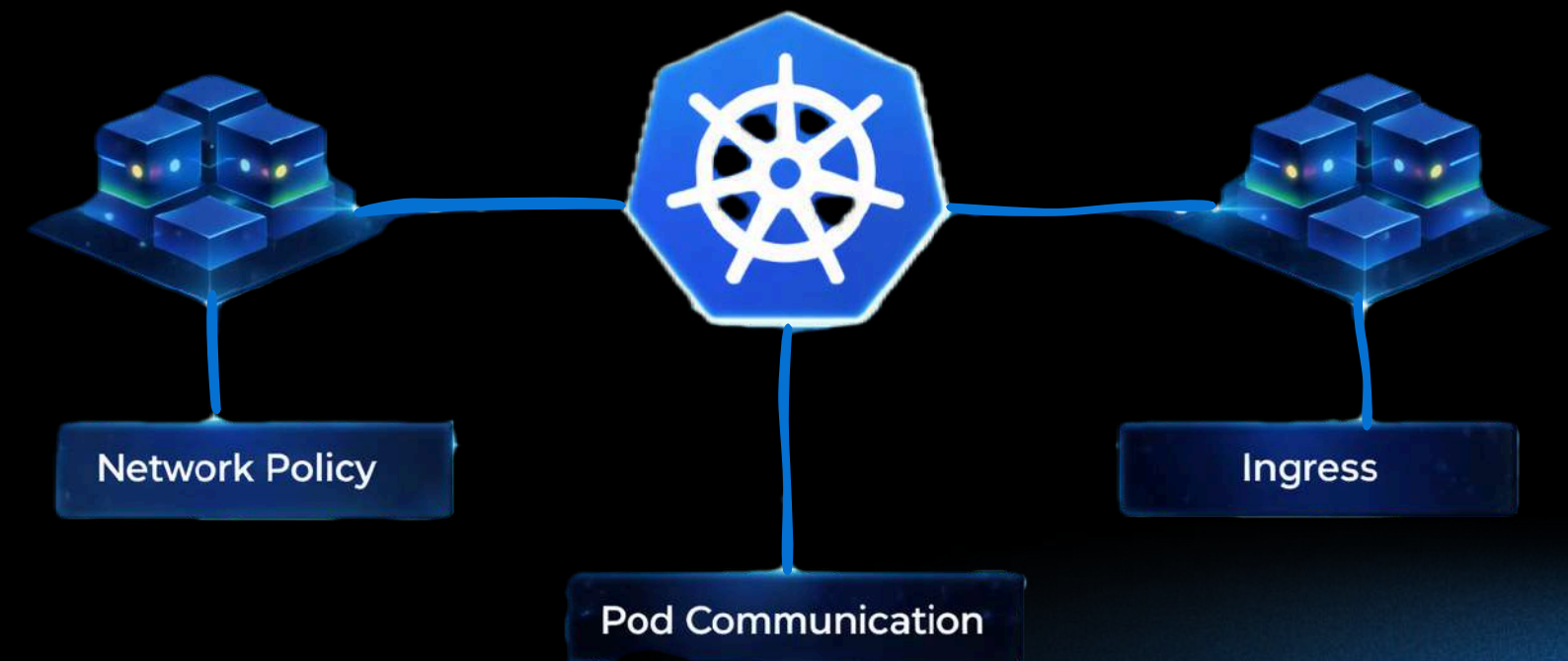
-1-

Why Kubernetes Networking Matters

Why Kubernetes Networking Matters

Kubernetes Networking is what connects your application to the world.

Kubernetes networking is what connects your application to the world. Applications don't fail because of code alone, in Kubernetes, most production issues are caused by traffic routing, service exposure, or misunderstood networking behavior. Understanding Kubernetes networking is what turns YAML files into real, reliable systems that users can actually reach and use.



-2-

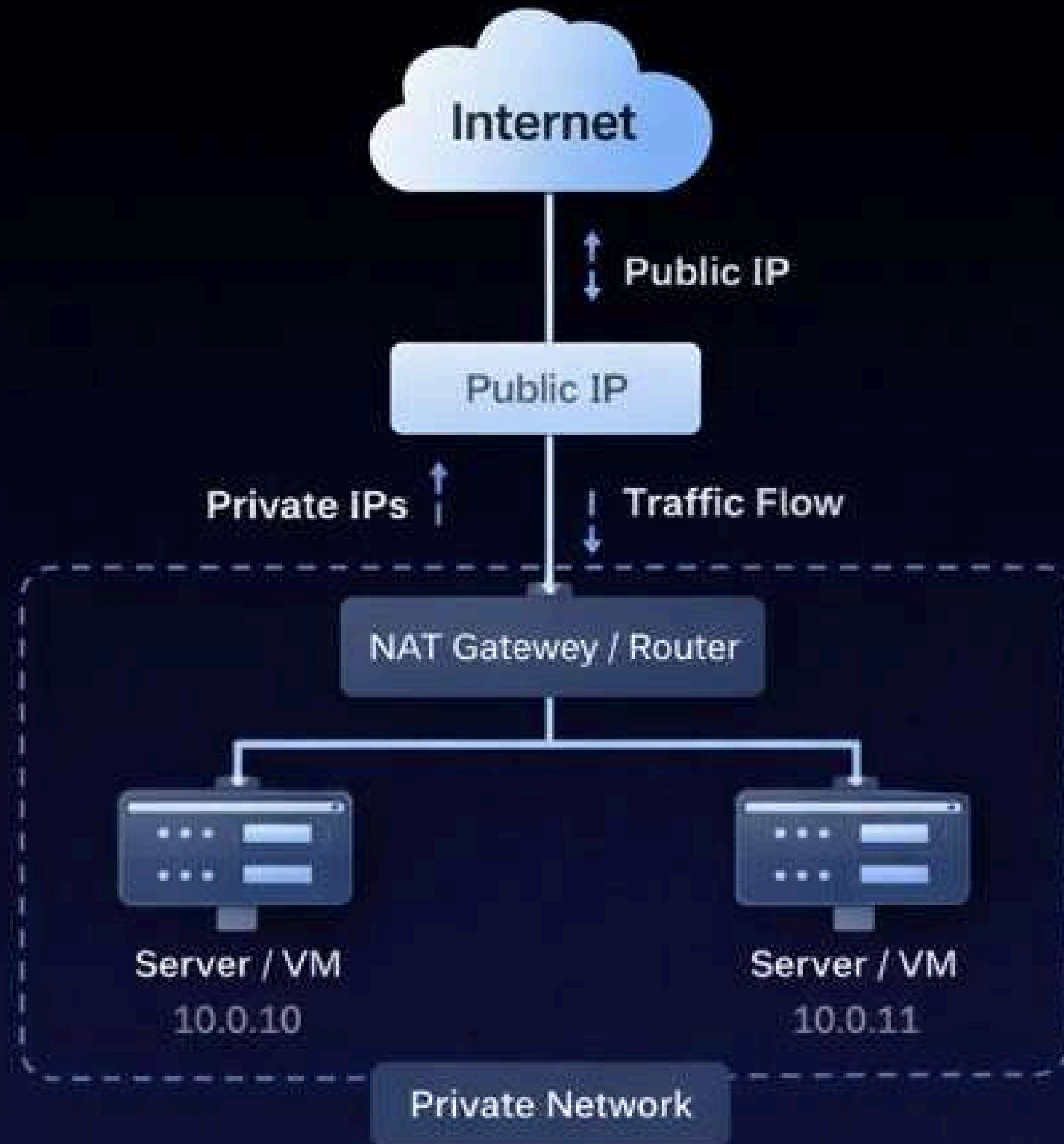
Networking Basics in Kubernetes

NAT = NETWORK ADDRESS TRANSLATION

In traditional infrastructure, applications usually live behind Network Address Translation (NAT). Servers or virtual machines use private IP addresses, while a single public IP address represents them to the outside world. NAT hides internal addresses, controls traffic flow, and requires port mapping or load balancers to expose services externally.

Traditional Networking & NAT

Traditional Networking & NAT



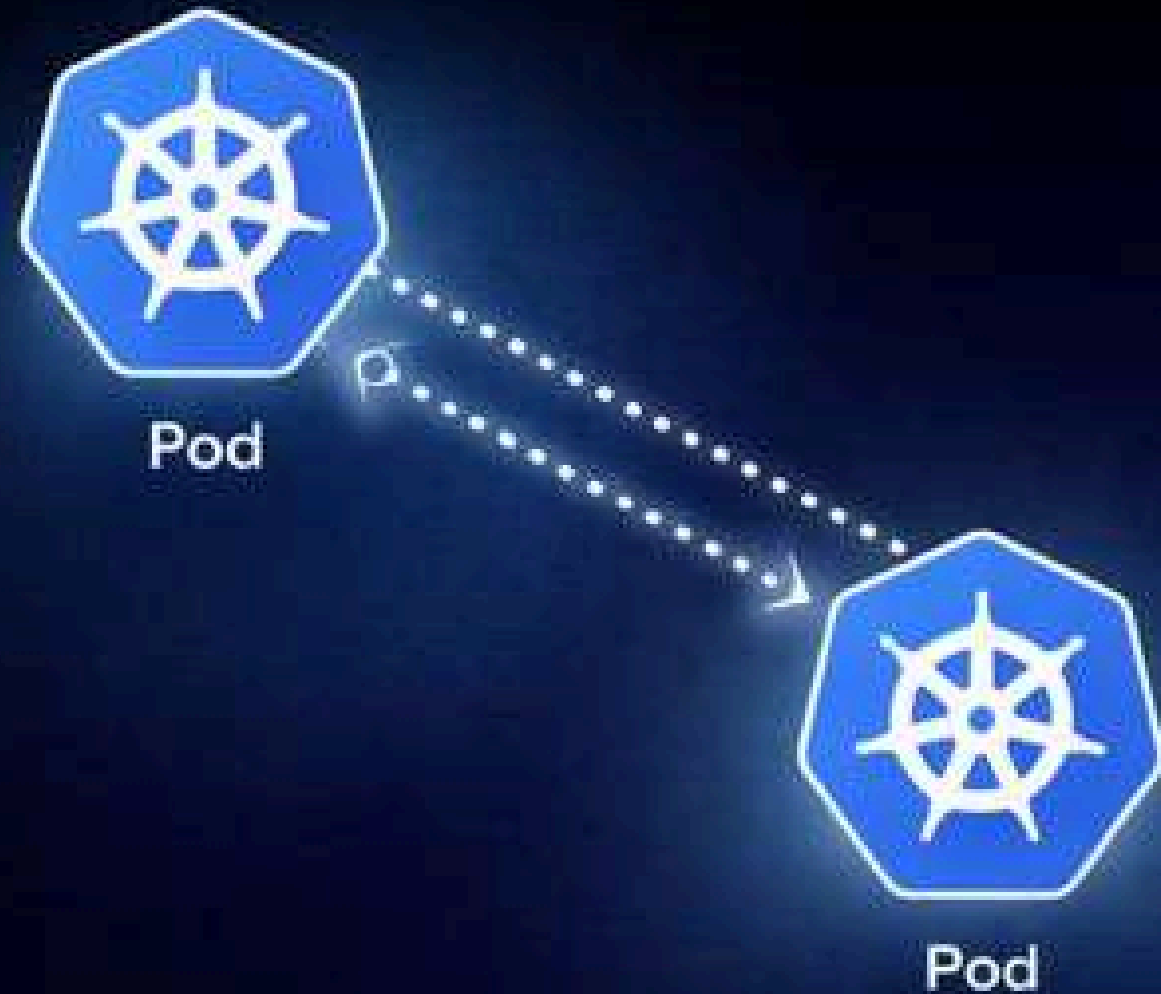
Kubernetes Networking Basics

In Kubernetes, every Pod is assigned its own IP address and can communicate directly with other Pods without using Network Address Translation (NAT). The cluster follows a flat networking model, where Pods can reach each other across nodes as if they are on the same network. This approach simplifies application-to-application communication, but it also introduces networking concepts that are very different from traditional VM- or server-based environments.

As a result, networking in Kubernetes is simpler at scale, but requires a different mental model.

-3-

Pod-to-Pod Communication



Pod-to-Pod Communication

In Kubernetes, Pods can communicate with each other directly using their IP addresses, even when they are running on different nodes. This direct connectivity is a core assumption of Kubernetes networking and is provided by the cluster's Container Network Interface (CNI). From the application's point of view, the network behaves as a single, shared network rather than isolated machines.

Pods can talk directly, but Pods are temporary.

-4.1-

Services Explained

Why Services Exist

Services give stability in a world where Pods are intentionally unstable.

Pods in Kubernetes are ephemeral; they can be created, destroyed, or rescheduled at any time, which means their IP addresses are not stable.

Services exist to provide a stable network endpoint that consistently routes traffic to the correct set of Pods. By abstracting Pod IPs, Services enable reliable communication between applications, even as the underlying Pods change.

Kubernetes Service Types

ClusterIP

ClusterIP is the default Service type and exposes an application only inside the Kubernetes cluster. It provides a stable internal IP and DNS name that other Pods can use to communicate with the Service. ClusterIP is commonly used for internal microservices that should not be accessible from outside the cluster.

NodePort

NodePort exposes a Service by opening a static port on each worker node. Traffic sent to any node's IP on that port is forwarded to the Service and then to the Pods. It is mainly used for testing or demos and is not recommended for production.

Kubernetes Service Types

LoadBalancer

LoadBalancer exposes a Service externally by provisioning a cloud provider's load balancer. It creates a public entry point that forwards traffic to the Service and its Pods. This approach is common in managed Kubernetes environments but can increase cost.

ExternalName

ExternalName maps a Service to an external DNS name instead of routing traffic to Pods. It does not create IPs or perform load balancing and is mainly used to integrate external services into the cluster.

-4.2-

Ingress & Ingress Controllers

Ingress

Ingress is a Kubernetes resource that defines how external HTTP and HTTPS traffic should be routed to Services inside the cluster. It allows you to expose multiple applications using a single entry point, based on hostnames or URL paths. Ingress works at the application layer and focuses on routing rules rather than direct traffic handling.



Ingress Controller

An Ingress Controller is the component that actually enforces the Ingress rules. It runs inside the cluster and watches for Ingress resources, then configures a proxy or load balancer to route traffic accordingly. Without an Ingress Controller, Ingress rules have no effect.

Ingress is just configuration; the Ingress Controller does the real work.



Ingress Today & Tomorrow

NGINX Ingress (Today)

Ingress is the traditional way to expose HTTP and HTTPS applications in Kubernetes. Controllers such as NGINX Ingress are widely used and still supported in production today. Ingress provides basic routing based on hosts and paths, but it has limitations in extensibility and role separation.

Gateway API (The Future)

The Kubernetes Gateway API is the next evolution of Ingress, designed to provide a more expressive, extensible, and role-oriented approach to traffic management. It introduces clear separation between infrastructure ownership and application routing, and is becoming the long-term direction for Kubernetes networking.

Ingress is not deprecated or disappearing tomorrow, but Gateway API is where Kubernetes networking is heading.

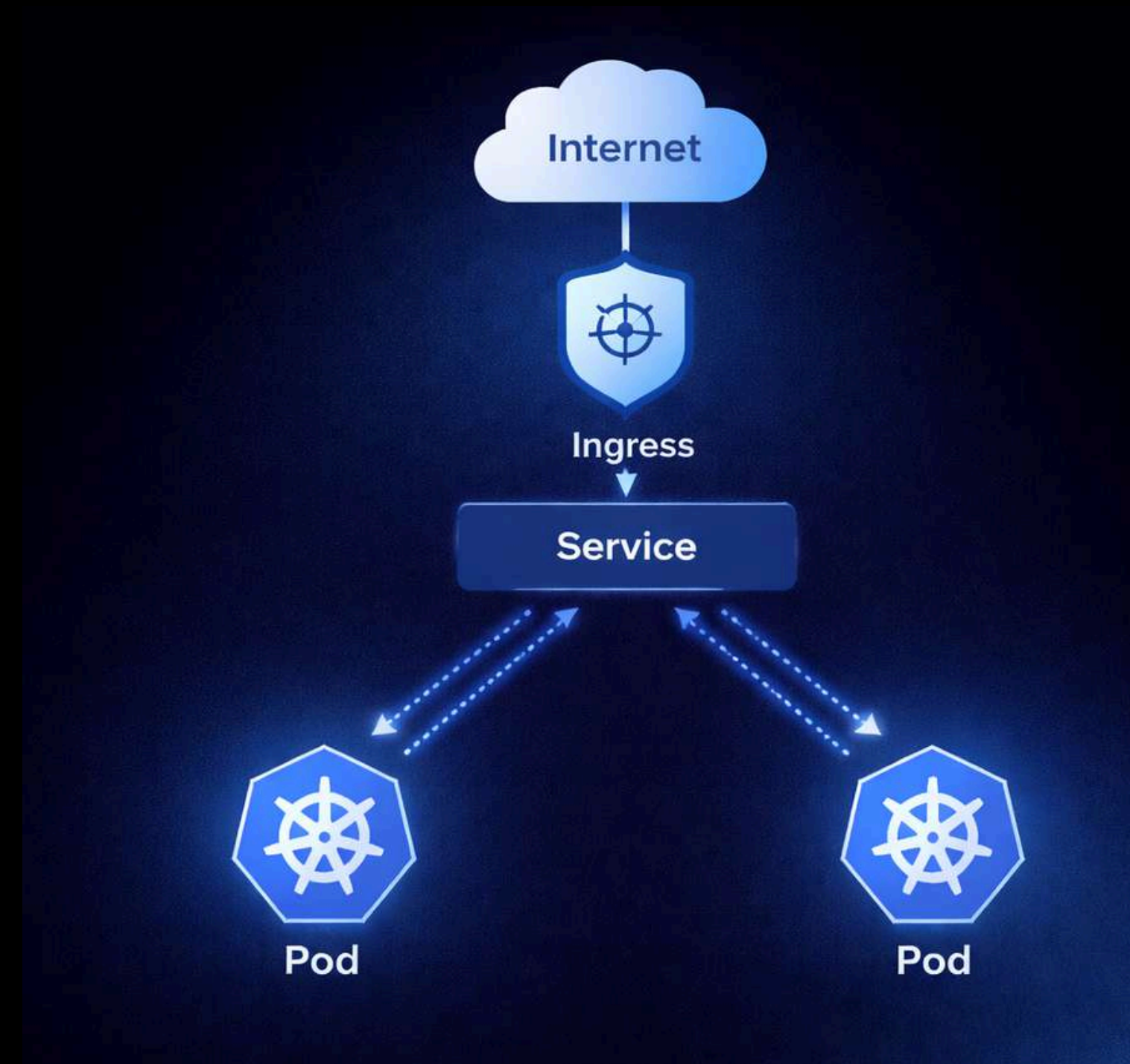
-5-

Traffic Flow: End-to-End

Traffic Flow: End-to-End

When a user sends a request to a Kubernetes application, the traffic enters the cluster through an external entry point such as an Ingress Controller or Gateway. The request is then routed to a Service, which selects one of the matching Pods and forwards the traffic to it. This flow abstracts infrastructure complexity while ensuring reliable and scalable communication.

Ingress decides where traffic goes, and Services decide which Pod gets it.



-6-

Common Networking Pitfalls

Common Kubernetes Networking Pitfalls



Kubernetes networking issues usually come from small configuration mistakes rather than complex failures. These problems are common when learning how Services and Ingress actually work together.

If traffic isn't working, it's usually the networking configuration, not the code.

Common Mistakes

Accessing Pods directly

Pod IPs change and should never be used as stable endpoints.

Using the wrong Service type

Choosing NodePort or LoadBalancer without understanding the impact can cause security and cost issues.

Common Mistakes

Expecting Ingress to work automatically

Ingress requires an Ingress Controller to handle traffic.

Confusing Services with Ingress

Services route traffic to Pods, while Ingress controls external access.

Common Mistakes

Hardcoding IP addresses

Kubernetes networking relies on DNS and Service discovery, not fixed IPs.

-7-

Practical Demo Overview

Demonstrated by
Maulik Shah

-8.1-

Key Takeaways

Key Takeaways

Kubernetes networking is built around stable abstractions over ephemeral components. By understanding how Pods, Services, and Ingress work together, you can design applications that are reliable, scalable, and easier to debug.

-8.2-

What's Next



What's Next

Now that we understand how traffic flows through a Kubernetes cluster, the next step is observing and troubleshooting that traffic. In Week 3, we'll focus on Kubernetes observability and how to monitor, measure, and debug applications running in production.

THANK YOU